

A sales assistant that answers from InsureX's own product documents, and knows when to say "I don't know"

Advanced RAG with LangGraph: grounded answers with page citations, lead capture through an MCP tool, and separate memory for every chat. Runs fully on a local machine.

91/100

questions right on the test

100%

right document retrieved

14/15

off-catalogue questions refused

2/2

bonus tasks done

Every requirement is covered, including both bonus tasks

What was asked, and where it lives in the code

Requirement	How it is built	Code	
Framework	LangChain for models, prompts and splitting; LangGraph state machine with 14 nodes, conditional routing and a retrieval cycle	agent.py	done
Vector DB	FAISS with cosine similarity, saved to disk; 5 PDFs → 152 chunks embedded with bge-m3 (Thai ⇄ English)	knowledge_base.py	done
Not-found handling	Similarity floor → LLM grader → rewrite the query → retry once → polite refusal. Never invents an answer	agent.py, prompts.py	done
Bonus 1: lead collection	Interest switches the agent into lead mode; name, occupation, income and phone are collected over several turns, validated, and saved through an MCP server into SQLite	leads.py, mcp_server.py	done
Bonus 2: session memory	LangGraph SQLite checkpointer , one thread per session; recent messages plus a running summary; chats can be renamed and deleted	sessions.py	done
Extras	React web chat with live streaming, a 100-question accuracy test, 36 automated tests, logs of every turn	frontend/, tests/	extra

Three things set it apart: it refuses instead of guessing, it reads real Thai insurance PDFs correctly, and every number in this deck comes from a repeatable test.

Five real InsureX product PDFs, one per category, mostly in Thai

Built from insurex.co.th and the partner product pages it links to: 5 documents, 152 searchable chunks

Document	What is inside	Chunks
Health	7 products (Easy E-Health, E-Health Mini, HB Plus, Prima Care...), benefit tables with up to 10 plans side by side	68
Accident	38 personal accident plans from 4 insurers, for kids, working age and seniors	49
Savings	3 savings plans: Easy E-Save 10/3, คั้มมั้งมี 18/9, คั้มออมสุข	14
Travel	Cover, optional extras and 28 FAQ answers	14
Pet	TIP Pet Lover plans S–XXL, conditions, exclusions, FAQ	7

How the PDFs were made

- 1 The brief's PDFs were not supplied, so each category page on **insurex.co.th** became one PDF.
- 2 Pages that build their content in the browser were rendered with **Playwright** in Edge: "show more" clicked until every plan appeared, every FAQ opened.
- 3 Plan counts were checked against the site's own totals.

Why it matters: real catalogues are harder than clean sample text. Big tables, many products per file and Thai script all broke the first version.

Three problems in the real PDFs, each found by testing and fixed in the loader

The PDF reader is where most of the accuracy is won or lost

1 · Thai text was broken

pypdf dropped tone marks and upper vowels:

เปี ย instead of เปี้ย (premium)

Fix: switched to **PyMuPDF**, and mapped legacy font glyphs (Unicode U+F700–F71A) back to standard Thai.

2 · Tables fell apart

Every table cell came out on its own line, so a premium lost its row and its plan.

รายปี 36,500 109,500 182,000 362,000

Fix: text that shares a **baseline** (within 3 pt) is joined, so a table row stays one line.

3 · Which product is this table for?

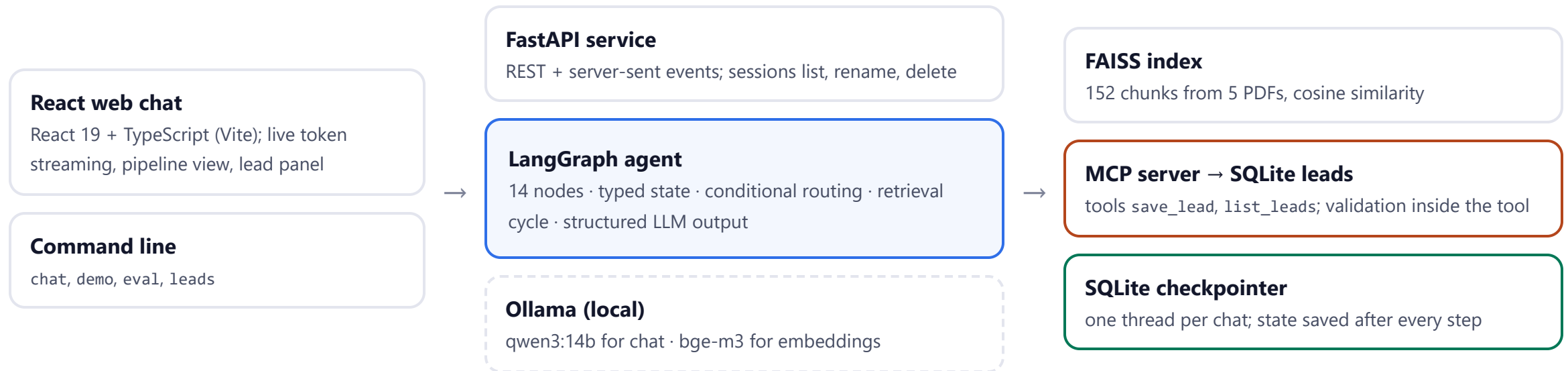
A health PDF holds 7 products, and a benefit table rarely repeats the product name. "แผน M 3,000" could be any of them.

Fix: every chunk is labelled "**document > section**", taken from the headings and carried across pages.

Result: English questions find Thai answers (multilingual bge-m3), and a benefit table stays linked to the right product. Each fix has its own unit test.

One agent, three stores, everything on the local machine

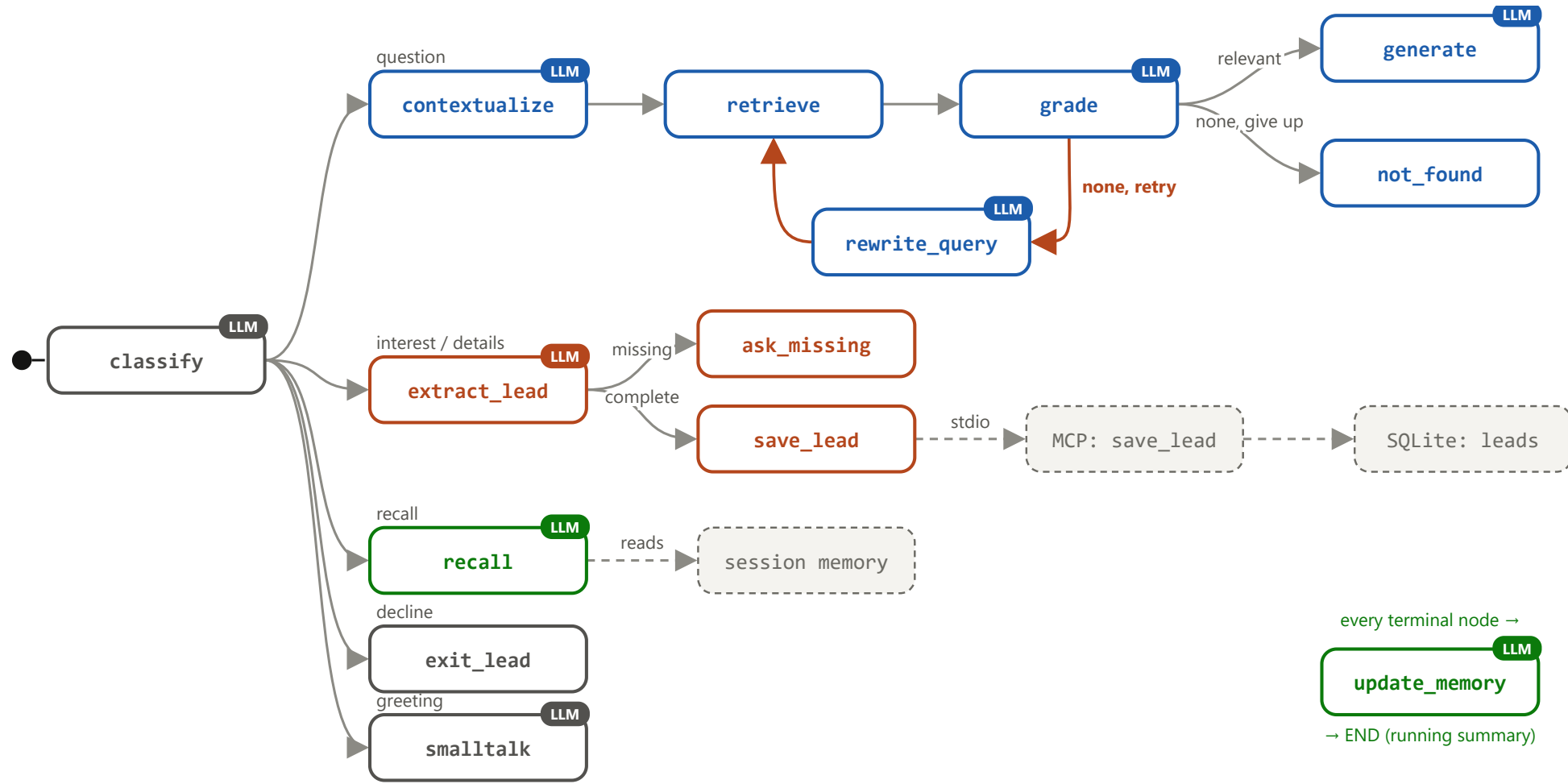
Customer data and the model never leave the computer; each part can be swapped through one setting or one class



Design choice: the LLM classifies, grades and writes, but the **control flow is code**. Every turn's path is logged and the rules can be unit-tested.

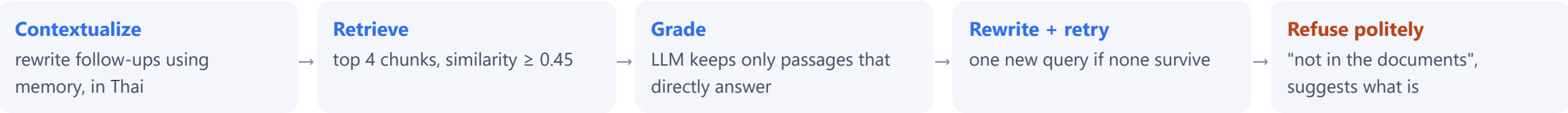
Every message is routed first, then takes one of three paths

blue answer from the documents **orange** collect a lead **green** memory · the LLM badge marks steps that call the model



Similar is not the same as relevant, so every answer passes a check first

An off-topic question can still score high similarity against insurance text, so an LLM grader checks the passages before any answer



Recorded: a product InsureX doesn't sell

Do you sell car insurance?

classify → contextualize → retrieve → grade → rewrite_query → retrieve → grade → not_found → update_memory

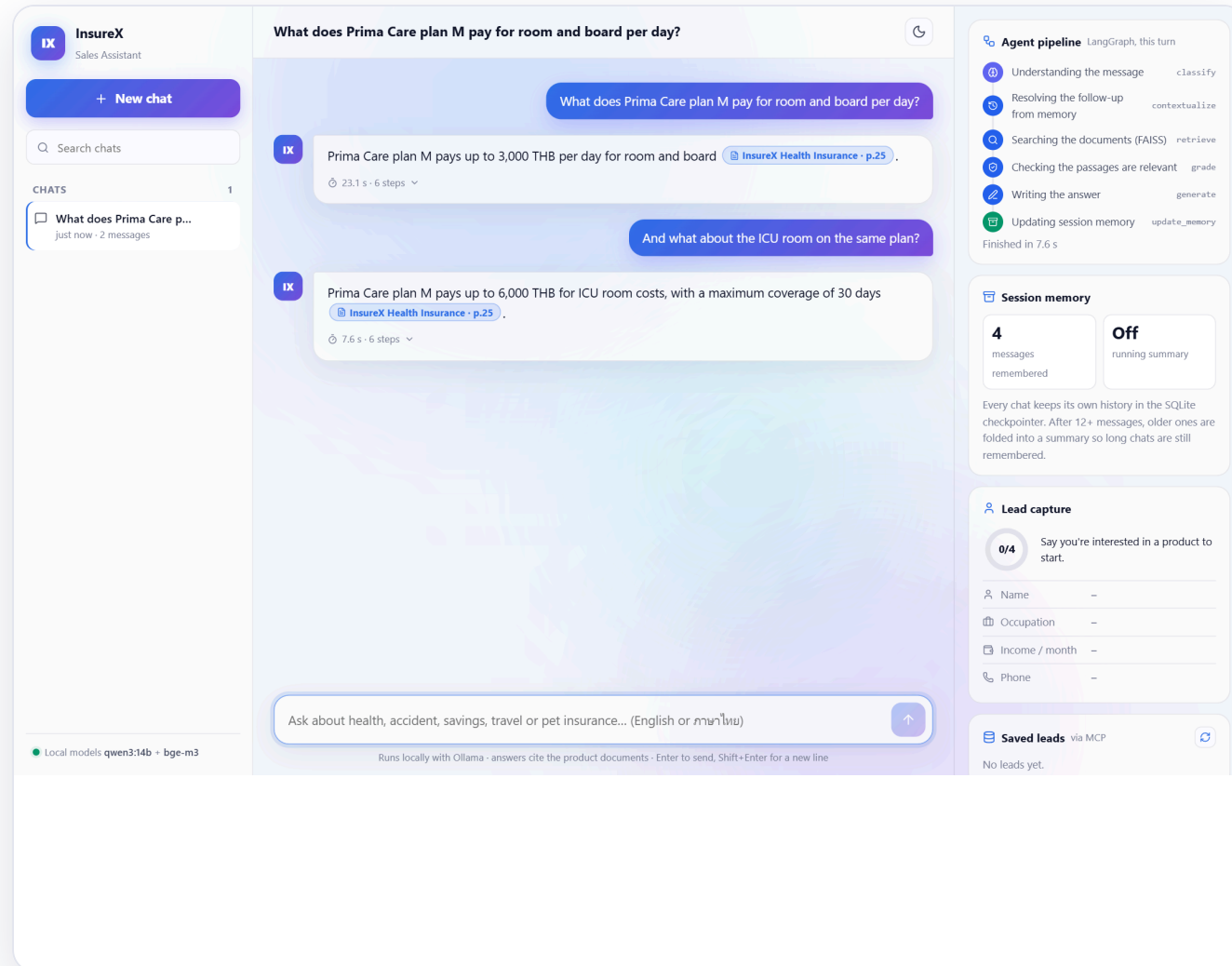
Sorry, I couldn't find this in the InsureX product documents, so I can't give a reliable answer. I can help with InsureX health, personal accident, savings, travel and pet insurance.

Rules that hold on every answer

- 1 No relevant passage → **no answer**. The graph, not the LLM, enforces it.
- 2 Every fact carries a **[file, page]** citation, shown as a source chip.
- 3 The retry counter lives in the state, so the cycle **cannot loop forever**.

English question, Thai document, cited answer, and the follow-up is understood

"The same plan" is resolved from memory; the right panel shows each LangGraph step as it runs



- 1 **Source chips** link each fact to the PDF and page (InsureX Health Insurance, p.25).
- 2 **Agent pipeline** lights up classify → contextualize → retrieve → grade → generate live.
- 3 Answers **stream** token by token; RAG answers take about 7.9 s on a local GPU.
- 4 Chats are listed, searchable, renamable and deletable, each with its own memory.

Interest turns the chat into lead capture, and bad data never reaches the database

Details are collected over several turns; a wrong phone number is rejected by the tool and asked for again

InsureX
Sales Assistant

+ New chat

Search chats

CHATS 1

What does Prima Care p...
just now · 5 messages

What does Prima Care plan M pay for room and board per day?

And what about the ICU room on the same plan?

IX Prima Care plan M pays up to 6,000 THB for ICU room costs, with a maximum coverage of 30 days
InsureX Health Insurance · p.25
7.6 s · 6 steps

I'm interested in Prima Care. Can an agent contact me?

IX Great to hear you're interested in Prima Care! So an agent can contact you, could you share your full name, your occupation, your monthly income (THB) and a contact phone number?
2.0 s · 4 steps

I'm Mai, a nurse, 30,000 a month, phone 12345

IX Sorry, some details don't look right. Could you check a contact phone number again?
3.5 s · 4 steps

sorry, 081-234-5678

IX Thank you, Mai! Your details are saved (lead #1). An InsureX agent will call you on 0812345678 about Prima Care shortly.
Lead #1 saved to SQLite via MCP
3.8 s · 4 steps

Ask about health, accident, savings, travel or products (โปรดอย่าลืมถามเกี่ยวกับสุขภาพ, อุบัติเหตุ, การออมเงิน, การเดินทาง หรือผลิตภัณฑ์ของเราด้วย)

Lead #1 saved via MCP

Runs locally with Ollama - answers cite the product documents - enter to send, Shift+Enter for a new line

Agent pipeline LangGraph, this turn

- Understanding the message classify
- Reading the customer's details extract_lead
- Saving the lead via the MCP tool save_lead
- Updating session memory update_memory

Finished in 3.8 s

Session memory

10 messages remembered

Off running summary

Every chat keeps its own history in the SQLite checkpoint. After 12+ messages, older ones are folded into a summary so long chats are still remembered.

Lead capture

4/4 Saved as lead #1 via the MCP tool.

Name	Mai	✓
Occupation	nurse	✓
Income / month	30,000 THB	✓
Phone	081-234-5678	✓

Saved leads via MCP

#1 Mai
Prima Care - nurse - 30,000 THB - 0812345678

- Lead mode persists** in the graph state, so a product question in the middle is answered and then the missing fields are asked again.
- The LLM extracts only what the customer typed into a **Pydantic** model.
- Validation is inside the MCP tool:** Thai phone format (normalised to 0XXXXXXXX), positive income, trimmed name. Any client gets the same rules.
- Recorded demo: **3 leads saved**, one after a rejected phone "12345".

Each customer gets a separate memory that survives restarts

LangGraph checkpoint in SQLite, with `thread_id = session_id`

Same session remembers

What's my name, and what did I ask about first?

Your name is Alice Wong, and you first asked about Prima Care plan M's coverage for room and board.

Another session knows nothing

What's my name? What did I ask you before?

I don't have any previous conversation history to reference.

Long chats stay cheap

- The last **8 messages** go to the LLM each turn.
- After **12** messages, older ones are folded into a **running summary** (`update_memory` node); the last 6 stay word for word.
- Deleting a chat erases its memory; saved leads stay as business records.

Survives a restart: the demo closes and re-creates the whole assistant, then Alice asks "Can you remind me what the room rate was?" and gets "The room rate for Prima Care plan M is up to 3,000 THB per day."

RESULTS

91 of 100 questions right, and the right document found every time

85 questions with a known answer (Thai and English) + 15 about products InsureX doesn't sell; each runs in a fresh session through the full graph

Overall

91%

91 / 100 questions

Retrieval

100%

right PDF in the top 4

Answers

91%

expected fact in the answer

Refusals

93%

14 / 15 off-catalogue questions

Correct answers by document



Reading it: the right PDF came back for every answerable question; most remaining errors are in **reading the right cell of a big table** (health and accident), not in finding the document.

The chunk size was chosen by the test, not by a rule of thumb

A quick retrieval sweep over 13 settings picked the candidates; each then ran all 100 questions end to end

Questions right, by chunk size / overlap (characters)



- 1 **1000 / 150** scored best: 91/100 (77/85 answers, 14/15 refusals).
- 2 Smaller chunks scored lower. The likely reason: at about 1000 characters a benefit-table block stays in one chunk with its plan headings.
- 3 Overlap stays at about 15%, so a fact on a chunk boundary is whole in at least one chunk.

Principle: every change in this project was checked against the same 100 questions, so improvements are numbers, not impressions.

The 9 misses are mostly table lookups, and each has a clear next step

Every failing question is listed in the presenter guide and in `logs/eval_report.md`

What goes wrong

- 1 **Wrong column or row in a wide table**, e.g. a premium read from the next plan or age band.
- 2 **A number from a neighbouring field**, e.g. the premium given when the fracture benefit was asked.
- 3 **Over-cautious refusals** on a waiting-period question whose answer was retrieved but not used.

What I would do next

- 1 **Parse tables into rows** (or a small SQL table) and let the agent query premiums directly.
- 2 **Hybrid search** (BM25 + vectors) and a **reranker** for exact plan names and numbers.
- 3 Grow the test set with the **sales team's real questions** and add an LLM judge for answer quality.

Safety holds even when it misses: wrong cells are the risk, not invented products, and refusals for unknown products held 14/15.

SUMMARY

Grounded, measurable and private

- Answers **only from InsureX's documents**, with page citations, and refuses when it can't.
- Reads **real Thai PDFs** correctly: text, tables, and which product each table belongs to.
- Captures **validated leads through MCP** and keeps a **separate memory per chat**.
- **91%** on a 100-question test, 36 automated tests, runs fully on a local machine.
- Next for production: table-aware retrieval, the sales team's questions, PDPA consent and CRM via the same MCP tool.

Thank you. Happy to run any question live or walk through the code.